SIMULATION ET OPTIMISATION **ISC - HEIG-VD**
J.-F. Hêche, E. Rappos **Automne 2026**

Travail pratique **Heuristique DSATUR**

Objectifs

Dans ce travail pratique vous devez programmer, tester puis comparer deux heuristiques de coloration séquentielle des sommets d'un graphe : l'heuristique « plus grand degré en premier » (LF, *Largest First*) de Welsh et Powell et l'heuristique DSATUR de Brélaz.

Travail de programmation à effectuer

Vous devez compléter les sources fournies qui contiennent le code de lecture et de gestion des graphes, ainsi que l'interface commune des algorithmes de coloration, en programmant les deux heuristiques ci-dessous. Tout votre code sera placé dans votre propre package, obtenu en renommant le package `groupX`. Les packages `graph` et `coloring` sont fournis et ne doivent pas être modifiés.

Code fourni

- ▷ `Graph` : graphe simple non orienté et immuable, stocké sous forme de listes d'adjacence (tableau de tableaux). Les sommets sont numérotés de 0 à $n-1$, avec $n \geq 1$.
- ▷ `GraphReader` : lecture d'un graphe depuis un fichier texte (`GraphFormatException` est levée si le fichier est mal formé).
- ▷ `ColoringAlgorithm` : interface contenant la méthode `color(graph)` que doit implémenter chaque heuristique.
- ▷ `Coloring` : solution renvoyée par un algorithme. Les couleurs sont des entiers consécutifs à partir de 0 et chaque sommet doit être colorié. La méthode `isProperFor(graph)` vérifie que la coloration est valide.
- ▷ `Main` : à modifier et adapter. La version fournie lit tous les fichiers `.txt` du répertoire `data` et exécute l'algorithme trivial donné en exemple (une couleur pour chaque sommet).

Heuristiques à programmer

Dans les deux heuristiques, dont les principes sont rappelés ci-dessous, les couleurs sont des entiers consécutifs en commençant à 0 et chaque sommet reçoit la plus petite couleur non utilisée par ses voisins déjà coloriés.

1. **LF (*Largest First*).** Les sommets sont traités dans l'ordre décroissant de leur degré, aucun départage particulier des égalités n'est demandé. Les complexités temporelle et spatiale doivent être en $O(n+m)$ dans le pire des cas (voir l'exercice 7 de la série 1 sur les heuristiques).

2. **DSATUR.** À chaque itération, on colore le sommet non colorié de plus grand degré de saturation. En cas d'égalité, on prend le sommet de plus grand degré dans le graphe initial (départage arbitraire en cas de double égalité). Les complexités temporelle et spatiale doivent toutes deux être en $O(n^2)$ dans le pire des cas.

Pour DSATUR, vous repartirez de l'algorithme développé dans le travail précédent. Si vous apportez des modifications à votre pseudocode, vous les expliquerez et les justifierez dans votre rapport.

Format des jeux de données

Chaque graphe est stocké dans un fichier texte (extension `.txt`) respectant le format suivant.

- ▷ La première ligne contient deux entiers : le nombre $n \geq 1$ de sommets suivi du nombre m d'arêtes du graphe. Les sommets sont numérotés de 0 à $n-1$.
- ▷ Les m lignes suivantes définissent les arêtes du graphe et contiennent chacune deux entiers correspondant aux numéros des extrémités de l'arête.

Le graphe est simple : un fichier contenant une boucle, une arête multiple ou un numéro de sommet hors de $\{0, \ldots, n-1\}$ est refusé à la lecture.

Travail de comparaison et d'analyse

Afin de comparer les performances des deux heuristiques vous devez les exécuter sur l'ensemble des jeux de données fournis. Pour chaque instance vous présenterez les informations et résultats suivants, dans un ou plusieurs tableaux :

- ▷ le nom de l'instance (nom du fichier, sans l'extension) ;
- ▷ le nombre n de sommets et le nombre m d'arêtes du graphe ;
- ▷ le degré maximum Δ du graphe ;
- ▷ la densité D du graphe : $D = \dfrac{2m}{n(n-1)}$ pour $n \geq 2$ ($D = 0$ pour $n = 1$) ;
- ▷ le nombre de couleurs utilisées par chaque heuristique puis le gain relatif de DSATUR sur LF ;
- ▷ le temps d'exécution pour chaque heuristique et l'augmentation relative entre DSATUR et LF.

Le temps à mesurer est celui de l'appel à `color(graph)` uniquement, le temps de lecture du fichier et celui de la vérification de la coloration ne doivent pas être pris en compte. Utilisez `System.nanoTime()` pour faire les mesures et indiquez les temps dans une unité adaptée (ms ou μs).

Remarque. Pour limiter l'influence de la machine virtuelle Java, effectuez quelques exécutions d'échauffement non comptabilisées, puis répétez la mesure plusieurs fois et reportez la moyenne (indiquez le nombre de répétitions effectuées dans votre rapport).

En argumentant à partir des résultats obtenus vous chercherez à répondre aux deux points suivants :

1. **Qualité des solutions.** En comparant le nombre de couleurs utilisées, y a-t-il une supériorité systématiquement de DSATUR sur LF ? Peut-on établir un lien entre certaines caractéristiques des graphes (taille, degré maximal, densité, ...) et les performances des deux heuristiques ?
2. **Temps d'exécution.** Les temps observés sont-ils cohérents avec les complexités théoriques attendues ($O(n + m)$ pour LF, $O(n^2)$ pour DSATUR) ? Le gain de DSATUR en nombre de couleurs justifie-t-il son surcoût en temps ? Pouvez-vous identifier des contextes où recommanderiez-vous l'une ou l'autre heuristique ?

Modalités et délais

- ▷ Le travail est à effectuer par groupe de deux (les mêmes que pour le travail précédent), en Java 21 ou 25.
- ▷ L'archive contenant les sources du projet ainsi que les fichiers de données des graphes à tester est disponible sur le site Cyberlearn du cours.
- ▷ Vous devez rendre une archive `zip` du répertoire `groupX` (au nom ajusté selon votre groupe), et uniquement de ce répertoire, contenant toutes vos sources, soigneusement documentées ainsi qu'un court rapport au format PDF contenant vos résultats et votre analyse.
- ▷ Vous devez rendre votre travail sur Cyberlearn au plus tard le **dimanche 6 décembre 2026** (avant minuit).